

C++ PROGRAMMING

# Templates Lab

Function Templates • Class Templates

Student Edition

|                 |                                                |
|-----------------|------------------------------------------------|
| <b>Topic</b>    | Generic Programming with Templates             |
| <b>Covers</b>   | Function Templates, Class Templates            |
| <b>Compiler</b> | g++ (C++98 or later — no special flags needed) |
| <b>Duration</b> | 60 – 90 minutes                                |

## Learning Outcomes

- ✓ Understand why templates reduce code duplication.
- ✓ Write function templates that work with any data type.
- ✓ Build class templates and use them with different types.
- ✓ Correctly place all code inside `main()` or other functions.

## SECTION 1 — Core Concepts

### What is a Template?

A template lets you write one function or class that works for **any data type**. Instead of writing separate functions for int, double, and string, you write it once and the compiler generates the correct version for each type you use.

#### Function Template — Basic Syntax

```
template <typename T>
T functionName(T parameter) {
// body – use T as the type
return parameter;
}
```

##### ● Key Points

**typename T** — T is a placeholder for any data type.

The compiler replaces T with the real type when you call the function.

You can name T anything, but T is the convention.

#### Class Template — Basic Syntax

```
template <typename T>
class ClassName {
T memberVariable;
public:
ClassName(T val) { memberVariable = val; }
T getValue() { return memberVariable; }
};
// To use a class template you MUST specify the type:
ClassName<int> obj1(10);
ClassName<double> obj2(3.14);
ClassName<string> obj3("hello");
```

##### ● Important Rule

Class templates require the type in angle brackets: MyClass<int>

Function templates deduce the type automatically from the arguments.

## SECTION 2 — Instructor Demo Programs

Run each demo live. Ask students: what happens if we change the type?

### Demo 1 — Function Template: findMax()

Returns the larger of two values. Works for int, double, char — any type that supports the > operator.

```
#include <iostream>
using namespace std;
template <typename T>
T findMax(T a, T b) {
    if (a > b)
        return a;
    else
        return b;
}
int main() {
    cout << findMax(10, 25) << endl; // 25 (int)
    cout << findMax(3.7, 2.1) << endl; // 3.7 (double)
    cout << findMax('z', 'a') << endl; // z (char)
    return 0;
}
```

#### ● Expected Output

25

3.7

z

### Demo 2 — Function Template: swapValues()

Swaps two variables of any type using a temporary variable.

```
#include <iostream>
using namespace std;
template <typename T>
void swapValues(T &a, T &b) {
    T temp = a;
    a = b;
    b = temp;
}
int main() {
    int x = 5, y = 10;
    cout << "Before: x=" << x << " y=" << y << endl;
    swapValues(x, y);
    cout << "After: x=" << x << " y=" << y << endl;
    double p = 1.1, q = 9.9;
    swapValues(p, q);
    cout << "p=" << p << " q=" << q << endl;
    return 0;
}
```

#### ● Expected Output

Before: x=5 y=10

After: x=10 y=5

p=9.9 q=1.1

### Demo 3 — Class Template: Box<T>

A simple container that stores one value of any type. Demonstrates a constructor, getter, and setter in a class template.

```
#include <iostream>
#include <string>
using namespace std;
template <typename T>
class Box {
    T value;
public:
    Box(T v) { value = v; }
    void setValue(T v){ value = v; }
    T getValue() { return value; }
    void display() {
        cout << "Box contains: " << value << endl;
    }
};

int main() {
    Box<int> b1(42);
    Box<double> b2(3.14);
    Box<string> b3("Hello");
    b1.display();
    b2.display();
    b3.display();
    b1.setValue(100);
    cout << "New value: " << b1.getValue() << endl;
    return 0;
}
```

### ● Expected Output

Box contains: 42

Box contains: 3.14

Box contains: Hello

New value: 100

## SECTION 3 — Student Practice Exercises

Complete each task in order. Save each in a separate .cpp file.

### ● Reminder — Always put code inside `main()` or another function!

Object creation and method calls must be inside `main()`.

Only class/function definitions go outside `main()`.

### 1 Function Template — `printTwice()`

#### Objectives:

- Write a function template `printTwice(T val)` that prints the value twice on separate lines.
- Call it in `main()` with an int, a double, and a string.

#### Hints:

- Use `cout` inside the function body.
- The function return type should be `void`.

### 2 Function Template — `findMin()`

#### Objectives:

- Write a function template `findMin(T a, T b)` that returns the smaller of two values.
- Test it in `main()` with integers, doubles, and characters.
- Print the result of each call.

#### Hints:

- Use the `<` operator to compare.
- The return type is `T`.

### 3

## Class Template — Pair<T>

### Objectives:

- Create a class template Pair<T> that stores two values of the same type.
- Add a constructor, getFirst(), getSecond(), and a display() method.
- In main(), create a Pair<int>, Pair<double>, and Pair<string>.
- Call display() on each.

### Hints:

- Store values as private members: T first, second;
- display() should print: First: X Second: Y

### 4

## ★ Challenge — Class Template: Calculator<T>

### Objectives:

- Write a class template Calculator<T> that stores two values a and b.
- Add methods: add(), subtract(), multiply() — each returns T.
- In main(), test with int values and double values.
- Print all three results for each type.

### Hints:

- Constructor: Calculator(T x, T y) sets a=x and b=y.
- Each method simply performs the operation on a and b and returns the result.

## SECTION 4 — Quick Reference

| What                     | Syntax                                                         | Example                            |
|--------------------------|----------------------------------------------------------------|------------------------------------|
| Function template        | <pre>template &lt;typename T&gt; T func(T a, T b)</pre>        | <pre>T findMax(T a, T b)</pre>     |
| Call a function template | <pre>func(val1, val2) (type is deduced)</pre>                  | <pre>findMax(10, 20)</pre>         |
| Class template           | <pre>template &lt;typename T&gt; class MyClass { T x; };</pre> | <pre>class Box { T value; };</pre> |
| Use a class template     | <pre>MyClass&lt;type&gt; obj;</pre>                            | <pre>Box&lt;int&gt; b(5);</pre>    |
| Pass by reference        | <pre>void swap(T &amp;a, T &amp;b)</pre>                       | <pre>swapValues(x, y)</pre>        |

### Common Mistakes to Avoid

- **Calling methods outside main()** — all statements must be inside a function.
- **Forgetting <type> for class templates** — write `Box<int> b(5)`, not `Box b(5)`.
- **Mixing types in one call** — `findMax(5, 2.5)` fails because `T` can't be both `int` and `double`. Use `findMax(5.0, 2.5)`.
- **Template definition in a separate .cpp** — keep templates in the same file as `main()` for now.

— End of Lab —